

元挂载器：模型挂载器的端到端优化

Yoonho Lee
Stanford

Roshen Nair
Stanford

Qizheng Zhang
Stanford

Kangwook Lee
KRAFTON

Omar Khattab
MIT

Chelsea Finn
Stanford

Project page w/ interactive demo: <https://yoonholee.com/meta-harness/>
Optimized harness: <https://github.com/stanford-iris-lab/meta-harness-tbench2-artifact>

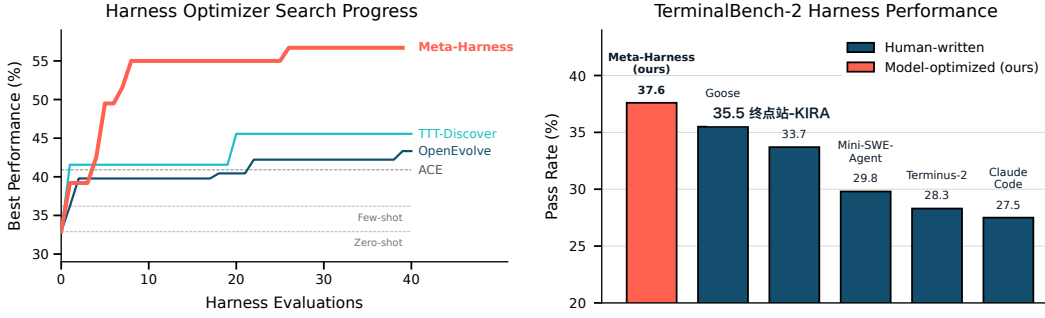


图 1: (左) 在文本分类任务上, 元挂载器优于此前最佳的手工设计挂载器 (ACE) 及现有文本优化器 (TTT-Discover、OpenEvolve), 仅需 4 次评估即达到次优方法的最终准确率。(右) 在 TerminalBench-2 上, 元挂载器优于所有已报告的 Claude Haiku 4.5 挂载器。

Abstract

大语言模型 (LLM) 系统的性能不仅取决于模型权重, 还取决于其挂载器: 即决定存储、检索和呈现给模型何种信息的代码。然而, 挂载器目前仍主要依靠手工设计, 而现有的文本优化器因过度压缩反馈而难以适配此场景: 它们无记忆、仅以标量分数为条件, 或将反馈限制为简短模板或摘要。本文提出元挂载器, 一种针对大语言模型应用在挂载器代码空间进行搜索的外循环系统。该系统采用智能体式提议器, 通过文件系统访问所有先前候选方案的源代码、分数和执行迹。在在线文本分类任务上, 元挂载器相较于最先进的上下文管理系统提升 7.7 个百分点, 同时减少 $4 \times$ 个上下文标记。在检索增强数学推理任务上, 单个发现的挂载器在 200 道国际数学奥林匹克级别题目上, 对五个留出模型的平均准确率提升 4.7 个百分点。在智能体编码任务上, 发现的挂载器在 TerminalBench-2 上超越最佳手工设计基线。这些结果共同表明, 更丰富地利用先前经验能够实现自动化挂载器工程。

1 引言

在固定的大语言模型 (LLM) 周围更换挂载器, 可在同一基准上产生 $6 \times$ 的性能差距 [47]。挂载器——即决定存储、检索和展示给模型何种信息的代码——其重要性往往不亚于模型本身。这种敏感性引发了人们对挂载器工程日益增长的兴趣, 即通过优化大语言模型周围的代码来提升整体系统性能的实践 [36; 21; 10; 9]。然而, 尽管挂载器工程至关重要, 它目前仍主要依赖人工: 从业者检查失败案例,

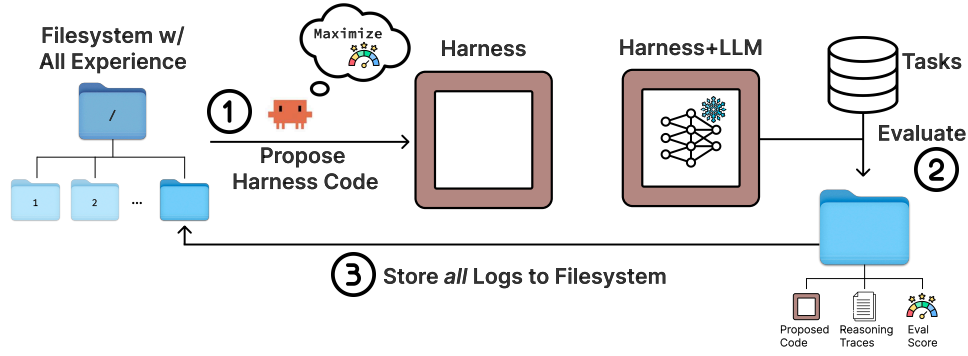


图 2：元框架搜索循环。(1) 智能体读取包含所有先前候选者源代码、执行迹和评分的文件系统，并提出新的框架。(2) 我们在评估任务上评估所提出的框架。(3) 所有日志（提议的代码、推理迹、评估分数）存储在新目录的文件系统中，循环重复。

Method	History	Log content	MTok/iter
OPRO [51]	Window	past (solution, score) pairs	0.002
TextGrad [53]	Last	textual feedback on current artifact	0.015
AlphaEvolve [35]	Window	program database + eval. scores	0.022
GEPA [1]	Summary	reflective feedback from rollout traces	0.008
Feedback Descent [26]	Summary	comparison + textual feedback	0.012
TTT-Discover [54]	Window	prev. solution fragment	0.026
Meta-Harness	Full	<i>all</i> logs and scores	10.0

表 1：文本优化方法及其设置的比较。每一行代表一种跨任务合并的方法。Mtok/iter 是我们对每篇论文中考虑的最大设置下，一次文本工件评估所生成的完整上下文的最佳估计。本文考虑的设置每次工件评估产生的上下文数量级更高。

调整启发式方法，并在少量设计上迭代。本文探讨该过程本身能否自动化。

一个自然的起点是近期关于文本优化的工作，因为框架工程同样涉及利用先前尝试的反馈迭代改进文本和代码工件 [38; 39; 35; 26; 1]。然而，这些方法与框架工程匹配不佳，因为它们通常采用短视界或高度压缩的反馈：有些仅以当前候选为条件 [31; 51; 53]，有些主要依赖标量评分 [35; 12]，还有些将反馈限制为简短模板或大语言模型生成的摘要 [1; 26]。这是一种务实的可扩展性选择，而非证明长程依赖无信息。框架在长视界上起作用：关于存储什么、何时检索、如何呈现的单一选择，可能影响许多推理步骤之后的行为。压缩反馈往往移除了追溯下游失败至早期框架决策所需的信息。在若干代表性文本优化器研究的任务中，每个优化步骤可用的上下文仅为 100 至 30,000 个标记（表 1），远低于框架搜索的诊断足迹。更广泛地，关于检索和记忆增强语言模型的研究表明，有用的上下文通常应自适应访问，而非单一地打包进一个提示 [28; 48; 37; 56]。

我们通过元框架（**Meta-Harness**）来解决这一局限性，这是一种通过端到端搜索来优化框架的智能体框架（图 2）。其提议者是一个编码智能体，即基于语言模型的系统，能够调用开发者工具并修改代码。选择编码智能体（而非原始大语言模型）至关重要，因为经验量很快会超出上下文限制，因此提议者必须决定检查什么，并通过与代码库的直接交互来验证编辑。其关键设计选择是通过文件系统暴露完整历史，从而能够对原始先前代码和执行轨迹进行选择性地诊断，而不是基于压缩的逐候选摘要进行优化。对于每一个先前的候选框架，文件系统存储源代码、评估分数和执行轨迹，提议者通过标准操作（如 `grep` 和 `cat`）检索这些内容，而不是将它们作为单个提示输入。实际上，在我们要求最高的设置中，提议者每次迭代平均读取 82 个文件，每步引用超过 20 个先前候选（附录 A）。在我们研究的设置中，

单次评估可产生多达 10,000,000 个标记的诊断信息，大约比先前文本优化设置中使用的最大反馈预算高出三个数量级（表 1）。

我们在在线文本分类、数学推理和智能体编码上评估了元框架。在在线文本分类上，元框架发现的框架比智能体上下文工程（ACE, Zhang 等人 [59]）提高了 7.7 个百分点，同时使用的上下文标记减少了 $4 \times$ 个，并且仅用四次提议就达到了次优文本优化器的最终性能（图 1）。在检索增强的数学推理上，一个发现的框架在 200 道国际数学奥林匹克级别的问题上，在五个保留模型上平均将准确率提高了 4.7 个百分点。在 TerminalBench-2 上，发现的框架超越了 Terminus-KIRA，并在所有 Haiku 4.5 智能体中排名第一。

2 相关工作

从高层次来看，元工具框架（Meta-Harness）将更广泛的文献中关于信用分配和元学习的思想 [40; 46; 3; 17; 44; 2] 引入了一个由编码智能体最新进展所促成的新机制中。该系统并非更新模型权重，而是在工具框架层面分配信用：它利用过去执行轨迹的经验，刻意推理哪些步骤和组件导致了失败，然后重写控制未来行为的外部代码。更具体地说，该方法处于几个近期研究方向的交汇点：它与自适应访问外部上下文、可执行代码搜索和文本优化的工作最为直接相关。

外部记忆与自适应访问。多项先前工作指出，将大型知识源或长输入视为语言模型自适应访问的外部资源，而非一次性消费，具有诸多益处。具体而言，检索增强生成 [28]、交错检索与推理 [48]、基于记忆的智能体 [37] 或递归语言模型 [56] 都是自适应访问外部上下文的机制。元工具框架采用了类似的访问模式，但应用于更具挑战性的工具框架工程场景，其中提议者选择性地检查包含代码、分数和执行轨迹的大型外部历史，以改进上下文管理程序本身。

可执行代码搜索。近期方法在可执行代码中搜索函数、工作流或智能体设计。早期工作提出在进化程序搜索中使用大型模型作为变异和交叉算子 [27]。后续方法在固定程序脚手架内进化指定函数 [39]，使用元智能体根据先前发现编程新智能体 [20]，或搜索智能体系统的工作流程图 [58]。另一条工作线搜索持续学习智能体的记忆设计，其中记忆在任务流中持续存在 [57; 50]。相比之下，元工具框架搜索特定领域的工具框架，包括提示构建、检索和状态更新策略，这些策略在任务之间重置。其外层循环刻意保持极简：不依赖固定脚手架、先前发现档案或持久记忆机制，而是赋予提议者对先前经验的无限制文件系统访问权限。这使得智能体能够决定检查哪些信息，并支持在完整工具框架实现上进行搜索，而非在预定义的上下文管理程序空间内搜索。

文本优化方法。元工具（Meta-Harness）也与诸如 ProTeGi、TextGrad、OPRO、GEPA、AlphaEvolve/OpenEvolve 和反馈下降（Feedback Descent）等方法密切相关，这些方法利用先前尝试的反馈迭代改进提示或其他文本工件 [38; 31; 53; 51; 1; 35; 43; 26]。然而，这些方法不太适合工具工程，因为工具工程优化的是一个完整的可执行程序，且相关的环境反馈分布在代码、分数和执行迹中，难以预先概括。元工具中的提议者并非仅对聚合分数或摘要做出反应，而是能够对失败示例及其执行迹进行推理，以提出有针对性的编辑。表 1 比较了这些论文与本文所考虑的问题规模，图 1 和图 4 展示了在我们的问题设置中与 OpenEvolve、GEPA 和 TTT-Discover 的直接比较。

3 元工具：用于优化工具的工具

本节描述元工具，即我们用于搜索任务特定工具的外循环过程。元工具基于这样的思想：工具优化受益于允许提议者通过文件系统访问选择性地检查先前的代码和执行迹，而不是从有损摘要或额外手工设计的搜索结构中进行优化。在高层次上，它反复提出、评估并记录新的工具。

元工具本身在广义上也是一种工具（因此得名），因为它决定了提议者模型在搜索过程中看到哪些信息。除非另有说明，我们使用“工具”一词指代被优化的任务特定程序。

目标。 工具是一个有状态程序，它包装语言模型并决定模型在每一步看到什么上下文。目标很简单：找到使底层模型在目标任务分布上表现最佳的工具。形式上，设 M 表示一个固定的语言模型， $\mathcal{X}$ 表示任务分布。对于工具 H 和任务实例 $x \sim \mathcal{X}$ ，我们执行一个 rollout 轨迹 $\tau \sim p_M(H, x)$ 。工具为 M 构建提示，模型做出响应，工具在每次交互后更新其状态。任务特定的奖励函数 $r(\tau, x)$ 对轨迹进行评分。工具优化的目标是找到使期望最终奖励最大化的工具：

$$H^* = \arg \max_H \mathbb{E}_{x \sim \mathcal{X}, \tau \sim p_M(H, x)} r(\tau, x),$$

当多个目标相关时（例如准确率和上下文成本），我们在帕累托支配下评估候选方案，并报告所得前沿。在实践中，这种搜索传统上由人类工程师和研究人员手动进行，他们通过手工迭代优化提示、上下文管理规则和工具使用逻辑。

元框架搜索循环。 元框架使用一个单一的编码代理提议器，该提议器可以访问一个不断增长的文件系统 $\mathcal{D}$ ，该文件系统充当其反馈通道¹。这里，编码代理是一个基于语言模型的系统，可以调用开发者工具并修改代码。与之前将改进逻辑外部化到手工设计的搜索循环中的系统不同，元框架将诊断和提议委托给编码代理本身：它决定检查哪些先前工件、解决哪些失败模式，以及是进行局部编辑还是更大幅度的重写。等价地，提议器不是一个在外部循环组装的固定提示上操作的原始下一个标记模型；它是一个在搜索过程中检索信息、导航先前工件并编辑代码的代理。每个被评估的框架贡献一个包含其源代码、分数和执行迹（如提示、工具调用、模型输出和状态更新）的目录。文件系统通常远大于提议器的上下文窗口，因此提议器通过终端工具（如 `grep` 和 `cat`）查询它，而不是将其作为单个提示摄取。在每次迭代中，提议器首先检查先前的代码、分数和执行迹，然后在生成新框架之前推理可能的失败模式。

元框架维护一个种群 $\mathcal{H}$ 和一个关于已评估框架的帕累托前沿，但不施加父代选择规则：提议器在提出新框架时可以自由检查任何先前框架及其执行迹。我们运行固定次数的迭代进化，并在帕累托前沿上进行最终的测试集评估。这种简单性是刻意的：通过将诊断和编辑决策留给提议器，而不是硬编码搜索启发式，元框架可以随着编码代理能力的增强而自动改进。提议器永远不会看到测试集结果；其唯一反馈来自搜索集，即在搜索过程中用于评估候选框架并生成改进反馈信号的任务实例子集，以及在这些搜索运行期间记录的执行迹。

代码空间搜索的优势。 驾驭优化发生在代码空间中，其中对检索、记忆或提示构建逻辑的微小改动可能在许多步骤之后影响行为，使得局部搜索启发式方法与问题匹配不佳。通过检查执行

¹ 基于早期探索，我们认为该工作流直到最近才变得实用，这得益于 2026 年初前后编码智能体能力的重大改进。

Algorithm 1 Meta-Harness outer loop over harnesses

```
1: Input: tasks  $\mathcal{X}$ , LLM  $M$ , proposer  $P$ , iterations  $N$ 
2: Initialize: population  $\mathcal{H}$  ▷ Initial set of valid harnesses
3: Initialize: filesystem  $\mathcal{D} \leftarrow \emptyset$  ▷ stores code, scores, traces
4: for  $H \in \mathcal{H}$  do
5:    $E_H \leftarrow \text{Evaluate}(H, M, \mathcal{X})$ 
6:    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, E_H)\}$ 
7: for  $t = 1 \dots N$  do
8:   Proposer  $P$  queries filesystem  $\mathcal{D}$  ▷ inspects prior harnesses and scores
9:   Proposer  $P$  proposes  $k$  new harnesses  $\{H_1, \dots, H_k\}$ 
10:  for  $H$  in  $\{H_1, \dots, H_k\}$  do
11:    if  $H$  passes interface validation then
12:       $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, \text{EVALUATE}(H, M, \mathcal{X}))\}$ 
13: return Pareto frontier of harnesses stored in  $\mathcal{D}$ 
```

迹，提议者通常能够推断出驾驭失败的原因以及哪些早期设计选择可能导致了失败，而不仅仅是失败本身，如附录 A 和 A.2 中的搜索轨迹所示。在那里，我们看到提议者广泛阅读先前的代码和日志，然后利用这些轨迹识别混淆的编辑、隔离可能的因果变更，并在反复回归后转向更安全的修改。因此，提议者可以在算法结构层面修改驾驭，从检索、记忆或提示构建逻辑的变更到完整的程序重写，而不是填充模板或应用预定义的变异算子。在实践中，它通常从一个强大的先验驾驭开始，但这是一种涌现策略，而非硬编码规则。尽管搜索空间很大，将驾驭表示为程序提供了一种自然的正则化偏差：编码模型倾向于提出连贯的算法，而不是脆弱、硬编码的解决方案，这使搜索偏向于可重用的上下文管理过程。该动作空间与前沿编码助手所训练的读写-执行 workflow 紧密对齐。

实用实现。在我们的实验中，每个驾驭都是一个单文件 Python 程序，用于修改特定任务的提示、检索、记忆和编排逻辑。在我们的实验中，提议者 P 是带有 Opus-4.6 的 Claude Code [4]。提议者由一个最小的领域特定技能指导，该技能描述了在哪里编写新的驾驭、如何检查先前的驾驭及其执行轨迹，以及它可以和不可以修改哪些文件。基础模型 M 因领域而异，并且始终冻结；详见第 4 节。在我们的实验中，一次典型的运行在 20 次迭代中评估大约 60 个驾驭。我们在附录 D 中提供了在新领域实现元驾驭的额外提示。

4 实验

我们在三个任务领域评估元框架（Meta-Harness）：在线文本分类、数学推理和智能体编码。在每个领域中，我们使用标准评估度量将搜索发现的框架与领域适用的基线进行比较。具体实验设置请参阅各小节。

我们与两大类方法进行比较。（1）人工设计策略：这些是为每个领域手工构建的框架，代表了当前上下文构建的最新技术。我们在相应小节中描述这些基线。（2）

程序搜索方法：这些方法利用反馈和奖励信号在候选框架上进行搜索，但设计用于比框架工程更小规模的场景。

4.1 在线文本分类

我们遵循 Zhang 等人 [59]；Ye 等人 [52] 的在线文本分类设置：大语言模型（LLM）一次接收一个带标签的示例，更新其记忆，并在留出的测试集上进行评估。我们使用 GPT-OSS-120B 作为大语言模型文本分类器，并考虑以下问题：

Harness	Datasets			Avg.	
	USPTO	S2D	Law	Acc	Ctx ↓
Zero-Shot	12.0	63.2	7.0	27.4	0
Few-Shot (8)	14.0	67.9	21.0	34.3	2.0
Few-Shot (32)	13.0	72.2	21.0	35.4	7.9
Few-Shot (all)	15.0	78.3	29.0	40.8	12.3
MCE [52] [†]	14.0	83.0	23.0	40.0	28.5
ACE [59] [†]	16.0	77.8	29.0	40.9	50.8
Meta-Harness	14.0	86.8	45.0	48.6	11.4

表 2：三个数据集上所有框架的测试集度量。 Ctx 表示上下文中额外的输入令牌（千）。†：来自 Ye 等人 [52]。↓ 的实现：越低越好。元框架在提高在线文本分类准确率的同时使用更小的输入上下文。

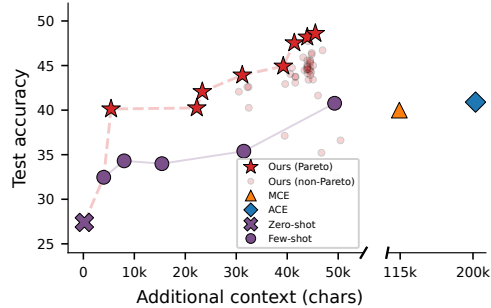


图 3：在线文本分类中准确率与上下文令牌的帕累托前沿。 元框架实现了比所有比较方法更强的准确率-上下文帕累托前沿。

设计一个用于文本分类的框架。我们使用三个数据集，这些数据集因其难度和领域多样性而被选中：**LawBench**（法律）[16] 从案件描述中预测刑事指控（215 个类别）；**Symptom2Disease**（S2D）[19] 从症状描述中预测疾病（22 个类别）；以及 **USPTO-50k** [41] 从产物分子中预测前体反应物（180 个类别）。我们从此设置中的主要基线框架初始化搜索种群 $\mathcal{H}$ ：零样本、少样本、ACE 和 MCE。我们进行了 20 次进化迭代，每次迭代产生两个候选框架，共产生 40 个候选框架。

与文本优化器的比较。 我们将元框架与优化文本的代表性方法进行比较。为了公平比较，我们使用相同的提议器配置（Opus-4.6 具有最大推理能力），仅基于搜索集性能选择候选框架，并将测试集保留到最终评估。由于评估是主要的计算瓶颈，我们为每种方法提供相同的提议框架评估预算。我们考虑以下比较点：

- **最佳 N 选：** 来自种子的独立样本，没有搜索结构；这是一个计算匹配的对照，用于检验搜索是否重要。
- **OpenEvolve** [43]：使用 LLM 变异对程序进行进化搜索。
- **TTT-Discover** [55]：我们仅使用其方法的文本优化组件，即通过 PUCT 重用规则进行提议选择。

在此设定下，元优化框架（Meta-Harness）与先前最优的文本优化器（OpenEvolve、TTT-Discover）在 $0.1\times$ 评估中发现了这一点，其最终准确率超过它们 10 个百分点以上（图 1 和表 4）。我们将这种加速归因于有意的设计选择，这些选择在外层循环上施加了最小必要结构（第 3 节）。特别是，元框架（Meta-Harness）利用文件系统保留完整的经验历史，并允许提议者检查任何必要内容，而开放进化（OpenEvolve）和测试时训练发现（TTT-Discover）的提议者输入则更加结构化且大幅受限，无法获得完整的文件系统访问权限。我们注意到，在线文本分类是我们研究的最小上下文设置（表 1），因此如果结构繁重的文本优化器在此处已经落后，它们的局限性在更困难的场景中可能只会进一步扩大。

元框架（Meta-Harness）速度提升 $10\times$ 倍，并收敛至更优框架

在此设定下，元优化框架（Meta-Harness）以少 $10\times$ 次完整评估的代价，与先前最优文本优化器（OpenEvolve、TTT-Discover）表现相当，且其最终准确率超出它们逾 10 个百分点。

为了分离提议者接口中哪些部分最为关键，我们在在线文本分类中比较了三种条件：仅分数条件、分数加摘要条件（提议者接收大语言模型生成的摘要但不接收原始轨迹），以及具备执行轨迹访问权限的完整元框架接口（表 3）。结果显示，完整接口具有显著优势：仅分数条件达到 34.6 的中位数准确率和 41.3 的最佳准确率，而分数加摘要条件达到 34.9 的中位数准确率和 38.7 的最佳准确率。相比之下，元框架

Method	Scores	Code	Summ.	Traces	Median $\uparrow$	Best Acc $\uparrow$	> ZS
Scores Only	✓	✓	✗	✗	34.6	41.3	26
Scores + Summary	✓	✓	✓	✗	34.9	38.7	23
Meta-Harness (full)	✓	✓	-	✓	50.0	56.7	39

表 3：在线文本分类中提议者可获取信息的消融研究。 > ZS：准确率超过零样本基线的运行次数。完整的元框架接口显著优于仅分数和分数加摘要的消融版本。获取原始执行迹是实现框架搜索的关键要素。

达到 50.0 的中位数准确率和 56.7 的最佳准确率，且其中位数候选方案甚至优于任一消融条件下找到的最佳候选方案。本文将此解释为：对执行迹（Execution Trace）的完全访问是接口中最重要的组成部分；摘要无法恢复缺失的信号，甚至可能因压缩掉具有诊断价值的细节而产生不利影响。

与最先进框架的比较。 本文的主要比较对象是针对该问题设置的手工设计框架：智能体上下文工程（Agentic Context Engineering, ACE, Zhang 等人 [59]），其利用反思性记忆管理随时间构建上下文；以及元上下文工程（Meta Context Engineering, MCE, Ye 等人 [52]），其维护并演化一个用于上下文构建的自然语言技能库。作为额外基线，我们评估了零样本提示和包含 $N \in \{4, 8, 16, 32, \text{全部}\}$ 示例的少样本提示。表 2 中的结果表明，元框架相比先前手工设计的框架有显著提升。所选元框架 2 达到 48.6% 的准确率，比 ACE 高出 7.7 个百分点，比 MCE 高出 8.6 个百分点。这些提升并非来自使用更多上下文：元框架仅使用 11.4K 上下文标记，而 ACE 为 50.8K，MCE 为 28.5K。

Method	Median	Best
GEPA [1]	32.6	40.2
Best-of-N	34.0	44.2
OpenEvolve [43]	39.1	43.3
TTT-Discover [55]	34.1	45.6
Meta-Harness	50.0	56.7

表 4：不同文本优化器所提框架的文本分类准确率（搜索集）。元框架在框架优化方面显著更为有效。

准确率与上下文权衡。 由于元框架对框架代码进行自由形式优化，我们可以表达对准确率和上下文成本的联合偏好，而非预先承诺单一标量目标。仅给定当前度量和期望的权衡，提议者能够在广泛的前沿范围内发现框架，从而在图 3 中产生平滑的准确率-上下文帕累托曲线。这使我们能够以可控方式用额外上下文换取更高的测试准确率，而非承诺单一手工设计的工作点。

分布外任务评估。 我们评估所发现的框架是否泛化到搜索期间未见过的全新数据集。我们考虑九个多样化的数据集，并在附录 C.1 中详细描述它们。所选元框架系统达到最佳平均准确率（73.1%），优于 ACE（70.2%）和所有少样本基线（表 5）。值得注意的是，我们观察到在 7 / 9 个任务中，朴素地添加超过 32 个少样本示例会损害性能。元框架在 6/9 个数据集上表现出最高性能，表明所发现的框架捕获了文本分类的普遍有效策略，而非过拟合搜索期间使用的特定数据集。

4.2 用于检索增强推理的框架

本文研究了一种用于奥林匹克数学求解的非标准设置：通过从大型语料库中检索示例来增强模型。从原理上讲，有充分理由预期检索有助于数学推理，因为解答往往共享可复用的证明模式，因此先前的推理轨迹包含模型在推理时可以利用的信息。

为简洁起见，本文略微重载了术语：在表格中，元框架（Meta-Harness）指发现的最佳框架，而在其他地方则指整个框架搜索过程。

Harness	SciC	FiNER	Amz5	FPB	GoEmo	Bank77	News	SciT	TwHate	Avg Acc	Ctx ↓
Zero-shot	32.7	56.0	52.7	90.0	42.0	80.7	84.7	89.3	75.3	67.0	-
Few-shot (8)	34.0	63.0	54.0	90.0	44.0	82.7	84.7	91.3	76.7	68.9	2.2
Few-shot (32)	38.7	62.0	53.3	90.7	43.3	86.0	85.3	90.7	76.7	69.6	5.2
Few-shot (all)	35.3	61.0	50.0	93.3	42.7	80.7	84.0	90.0	76.7	68.2	7.4
ACE [59]	40.7	74.0	48.0	96.7	44.0	83.3	86.0	90.7	68.7	70.2	11.7
Meta-Harness	53.3	67.0	60.0	94.0	46.0	82.7	86.7	91.3	77.3	73.1	7.3

表 5：分布外文本分类数据集评估。本文报告了每个数据集的测试准确率以及所有九个数据集的平均额外上下文标记数。元框架在这九个先前未见过的任务上比次优方法高出 2.9 个百分点。

Method	GPT-5.4n	GPT-5.4m	Gem-3.1FL	Gem-3F	GPT-20B	Avg.
No Retriever	23.0	28.8	28.6	42.6	47.6	34.1
Dense Retrieval ($k=1$)	27.1 (+4.1)	24.5 (-4.3)	31.3 (+2.7)	42.3 (-0.3)	46.9 (-0.7)	34.4 (+0.3)
Dense Retrieval ($k=5$)	31.1 (+8.1)	28.3 (-0.5)	37.1 (+8.5)	47.2 (+4.6)	46.7 (-0.9)	38.1 (+4.0)
Random Few-shot	23.1 (+0.1)	24.5 (-4.3)	31.0 (+2.4)	40.4 (-2.2)	41.8 (-5.8)	32.2 (-1.9)
BM25 Retrieval	30.2 (+7.2)	29.2 (+0.4)	32.8 (+4.2)	46.6 (+4.0)	48.9 (+1.3)	37.5 (+3.4)
Meta-Harness	31.7 (+8.7)	30.4 (+1.6)	34.9 (+6.3)	46.3 (+3.7)	50.6 (+3.0)	38.8 (+4.7)

表 6：在 200 道国际数学奥林匹克级别数学题上的检索增强求解。本文展示了每道题三次采样的平均通过率@1，括号内为相对于基线的绝对提升。所发现的元框架检索策略在所有五个留出模型上均提升了这些国际数学奥林匹克级别问题的推理能力，相比无检索器平均提升 4.7 个百分点。

然而，检索尚未成为该设置中的标准组成部分，先前的研究表明，与更依赖事实的领域相比，检索在推理密集型数学基准上的成功程度要低得多 [42; 49; 6]。困难在于，朴素的检索很少能以正确的形式浮现出正确的轨迹。这表明成功较少取决于添加检索本身，而更多取决于发现正确的检索策略。本文并未手工设计该策略，而是为元框架提供一组困难的奥林匹克问题，并允许检索行为本身从搜索中涌现。

检索语料库包含 $\geq$ 来自八个开源数据集的 500,000 个已解决问题。我们仔细对其进行了去重和去污染处理，针对评估基准和搜索集均进行了处理，确认在基于字符串的过滤下，留出问题没有精确前缀匹配，并对留出示例的 BM25 检索结果进行了人工检查（附录 C.2）。我们使用 Meta-Harness 在包含 250 个奥数难度数学问题（OlympiadBench + Omni-MATH hard）的搜索集上优化了 40 次迭代的 harness，产生了 109 个候选检索 harness。我们初始化搜索种群 $\mathcal{H}$ ，使用该设置下的主要基线 harness：零样本、少样本和 ACE。我们基于 GPT-OSS-20B 的搜索集性能选择单个 harness（附录 B.2）。我们在 200 个以前未见过的 IMO 级别问题上评估该 harness，这些问题来自 IMO-AnswerBench、IMO-ProofBench 和 ArXivMath [30; 6]。除了 GPT-OSS-20B，我们还在搜索过程中未见的四个模型上评估相同的检索 harness：GPT-5.4-nano、GPT-5.4-mini、Gemini-3.1-Flash-Lite 和 Gemini-3-Flash。我们遵循先前工作的标准评估协议 [30]，并报告每个问题三次采样的平均准确率。

结果。表 6 将发现的 harness 与无检索、使用独立嵌入模型 text-embedding-3-small 的稠密检索、随机少样本提示和 BM25 检索进行了比较。相比之下，Meta-Harness 完全在代码空间中操作，基于与稀疏基线相同的基于 BM25 的词法检索栈，而不是引入额外的稠密编码器。发现的检索 harness 在所有五个留出模型上均优于无检索基线，平均提升 4.7 个百分点。它还在平均上匹配或超过了最强的固定基线，总体上比 BM25 检索高出 1.3 个百分点，同时避免了在多个模型上观察到的稠密检索和随机少样本提示的性能下降。

Meta-Harness Improves Reasoning on IMO-Level Math Problems

在检索增强的数学推理中，单个发现的检索 harness 在五个留出模型上迁移，与无检索相比平均准确率提升 4.7 个百分点，并在比较的方法中取得了最强的总体平均性能。

4.3 在 TerminalBench-2 上评估智能体编码 harness

TerminalBench-2 [33] 在 89 项具有挑战性的任务上评估大语言模型智能体，这些任务要求在复杂依赖关系下进行长时程、完全自主的执行，并需要大量的领域知识。先前的工作表明，智能体框架的选择对该基准上的性能有很大影响。我们从两个强大的开源基线 Terminus 2 [33] 和 Terminus-KIRA [25] 开始搜索。在本实验中，我们在相同的 89 项任务基准上进行搜索和最终评估。我们将该基准用作发现性问题 [54]，其目标是发现一种框架配置，以在一个困难且公开竞争的基准上提升性能。这是标准做法：公开的文档已经描述了在 TerminalBench 本身上反复进行的基准特定框架迭代 [18; 34; 25]，而且该基准规模小且成本高，引入单独的分割会实质性地削弱搜索信号。此外，我们通过人工检查和基于正则表达式的审计来检查过拟合，以检测任务特定字符串是否泄漏到演化后的框架中。我们注意到，尽管得到的框架专门针对 TerminalBench-2 场景，但从单条指令自主完成困难的长期任务是核心能力，而且该基准包含许多前沿模型和重度工程化框架都难以应对的任务。

结果。我们在表 7 中报告了完整基准上的结果，在两个基础模型上进行了评估：克劳德·欧普斯 4.6 (Claude Opus 4.6) 和克劳德·俳句 4.5 (Claude Haiku 4.5)。在欧普斯 4.6 上，元框架 (Meta-Harness) 发现了一个达到 76.4% 通过率的框架，超过了手工设计的终点站-基拉 (Terminus-KIRA, 74.7%)，并在终端基准 2 (TerminalBench-2) 排行榜的所有欧普斯 4.6 智能体中排名第二。唯一得分更高的欧普斯 4.6 智能体是锻造代码 (ForgeCode, 81.8%)；然而，我们无法从公开可用的代码中复现他们报告的结果，这表明他们的排行榜得分依赖于已发布存储库之外的组件。在较弱的俳句 4.5 模型上，改进更大：元框架达到 37.6%，比次优报告智能体 (鹅, Goose, 35.5%) 高出 2.1 个百分点。终端基准 2 是一个竞争激烈的基准，多个团队直接针对它进行优化，因此自动搜索方法能够在这一前沿取得收益，对于长时程文本优化循环来说是令人鼓舞的。

提议者的定性行为。框架搜索轨迹有助于解释元框架为何能取得这些收益；我们在附录 A 中提供了详细总结。在早期迭代中，提议者将合理的结构修复与提示模板编辑相结合，并观察到两个候选方案均出现退化。随后，它明确假设退化受到共享提示干预的混淆，将结构变化与提示重写分离，并最终转向更安全的增量修改，该修改成为本次运行中的最佳候选。这提供了定性证据，表明文件系统访问使提议者能够足够详细地检查先前经验，从而形成因果假设并相应地修订框架。

Harness	Auto	Pass (%)
Claude Opus 4.6		
Claude Code	×	58.0
Terminus 2	×	62.9
Mux	×	66.5
Droid	×	69.9
TongAgents	×	71.9
MAYA-V2	×	72.1
Terminus-KIRA	×	74.7
Capy	×	75.3
ForgeCode	×	81.8
Meta-Harness	✓	76.4
Claude Haiku 4.5		
OpenHands	×	13.9
Claude Code	×	27.5
Terminus 2	×	28.3
Mini-SWE-Agent	×	29.8
Terminus-KIRA	×	33.7
Goose	×	35.5
Meta-Harness	✓	37.6

表 7：TerminalBench-2 上的通过率。其他结果来自官方排行榜。在这个竞争性任务上，元框架在所有 Opus-4.6 智能体中排名第 2，在所有 Haiku-4.5 智能体中排名第 1。

Meta-Harness Surpasses Hand-Engineered Agents on TerminalBench-2

在终端基准 2 上，元框架自动发现了在欧普斯 4.6 上超越终点站-基拉、并在所有俳句 4.5 智能体中排名第一的框架。

5 讨论

除了超越现有框架之外，元框架（Meta-Harness）还具有若干实用优势。发现的框架能够泛化到分布外分类数据集（表 5）以及数学场景中未见的基础模型（表 6）。一次搜索运行在几个小时的墙钟时间内即可完成，却能产生可读、可迁移的策略，这些策略可以跨模型复用，包括未来更强的模型。代码空间中的过拟合也更具可检查性：脆弱的条件链或硬编码的类别映射在检查时是可见的，而权重空间中的过拟合则不然。更广泛地说，我们的结果表明，元框架的主要优势不仅在于对代码的搜索，还在于有选择地访问先前诊断经验的搜索。提议者不受限于标量奖励或固定摘要；它可以检查原始代码、执行轨迹和先前的失败，然后利用这些信息形成并检验关于需要改变什么的假设。附录 A.2 中的定性搜索轨迹直接展示了这一行为。

我们的发现反映了机器学习中反复出现的模式 [45]：一旦搜索空间变得可访问，更强的通用智能体就能超越手工设计的解决方案。未来工作的一个自然下一步是共同演化框架和模型权重，让策略塑造模型所学内容，反之亦然。虽然我们在三个不同领域进行了评估，但我们的实验表明，框架搜索可以与一个特别强大的编码智能体提议者（Claude Code）配合使用；关于该效果在不同提议者智能体之间如何变化的更广泛研究仍有待未来工作。

致谢

我们感谢 KRAFTON AI 提供应用程序编程接口 (API) 额度支持。本工作得到 OpenAI、KFAS 和 Schmidt Sciences AI2050 的支持。我们感谢 Anikait Singh 和 Jubayer Ibn Hamid 提出的宝贵反馈和建议, 以及 Sienna J. Lee 在本工作早期阶段耐心倾听 YL 尚未成熟的想法。

参考文献

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gpa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [2] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models, 2023. URL <https://arxiv.org/abs/2211.15661>.
- [3] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. *Advances in neural information processing systems*, 29, 2016.
- [4] Anthropic. Claude code: An agentic coding tool. <https://www.anthropic.com/claude-code>, 2025.
- [5] Anthropic and community contributors. agentskills/agentskills. GitHub repository <https://github.com/agentskills/agentskills>. Specification and documentation for Agent Skills, accessed March 27, 2026.
- [6] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- [7] Francesco Barbieri, Jose Camacho-Collados, Leonardo Neves, and Luis Espinosa-Anke. Tweeteval: Unified benchmark and comparative evaluation for tweet classification, 2020. URL <https://arxiv.org/abs/2010.12421>.
- [8] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, June 2023. ISSN 2475-1421. doi: 10.1145/3591300. URL <http://dx.doi.org/10.1145/3591300>.
- [9] Birgitta Böckeler. Harness engineering. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>, March 2026. martinfowler.com.
- [10] Can Bölük. I improved 15 LLMs at coding in one afternoon. only the harness changed. <https://blog.can.ac/2026/02/12/the-harness-problem/>, February 2026.
- [11] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders, 2020. URL <https://arxiv.org/abs/2003.04807>.
- [12] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, et al. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- [13] Harrison Chase. Langchain, October 2022. URL <https://github.com/langchain-ai/langchain>. Software, released 2022-10-17.
- [14] Arman Cohan, Waleed Ammar, Madeleine van Zuylen, and Field Cady. Structural scaffolds for citation intent classification in scientific publications, 2019. URL <https://arxiv.org/abs/1904.01608>.

-
- [15] Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. Goemotions: A dataset of fine-grained emotions, 2020. URL <https://arxiv.org/abs/2005.00547>.
- [16] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Alan Huang, Songyang Zhang, Kai Chen, Zhixin Yin, Zongwen Shen, et al. Lawbench: Benchmarking legal knowledge of large language models. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 7933–7962, 2024.
- [17] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, 2017.
- [18] ForgeCode. Benchmarks don’t matter, 2025. URL <https://forgecode.dev/blog/benchmarks-dont-matter/>.
- [19] Gretel AI. Symptom to diagnosis dataset. <https://huggingface.co/datasets/gretelai/symptom.to.diagnosis>, 2023. Accessed: 2026-01-22.
- [20] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- [21] Anthropic Justin Young. Effective harnesses for long-running agents. <https://anthropic.com/engineering/effective-harnesses-for-long-running-agents>, November 2025. Anthropic Engineering Blog.
- [22] Phillip Keung, Yichao Lu, György Szarvas, and Noah A. Smith. The multilingual amazon reviews corpus, 2020. URL <https://arxiv.org/abs/2010.02573>.
- [23] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- [24] Tushar Khot, Ashish Sabharwal, and Peter Clark. Scitail: A textual entailment dataset from science question answering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi: 10.1609/aaai.v32i1.12022. URL <https://ojs.aaai.org/index.php/AAAI/article/view/12022>.
- [25] KRAFTON AI and Ludo Robotics. Terminus-kira: Boosting frontier model performance on terminal-bench with minimal harness, 2026. URL <https://github.com/krafton-ai/kira>.
- [26] Yoonho Lee, Joseph Boen, and Chelsea Finn. Feedback descent: Open-ended text optimization via pairwise comparison. In *arXiv preprint arXiv:2511.07919*, 2025.
- [27] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models, 2022. URL <https://arxiv.org/abs/2206.08896>.
- [28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [29] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. Finer: Financial numeric entity recognition for xbrl tagging. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4419–4431. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.303. URL <http://dx.doi.org/10.18653/v1/2022.acl-long.303>.

-
- [30] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- [31] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [32] Pekka Malo, Ankur Sinha, Pyry Takala, Pekka Korhonen, and Jyrki Wallenius. Good debt or bad debt: Detecting semantic orientations in economic texts, 2013. URL <https://arxiv.org/abs/1307.5336>.
- [33] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- [34] Jack Nichols. How we scored #1 on terminal-bench (52%), Jun 2025. URL <https://www.warp.dev/blog/terminal-bench>.
- [35] Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [36] OpenAI. Harness engineering: leveraging Codex in an agent-first world. <https://openai.com/index/harness-engineering/>, February 2026. OpenAI Blog.
- [37] Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. 2023.
- [38] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. *arXiv preprint arXiv:2305.03495*, 2023.
- [39] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [40] Jurgen Schmidhuber. A neural network that embeds its own meta-levels. In *IEEE International Conference on Neural Networks*, 1993.
- [41] Nadine Schneider, Nikolaus Stiefl, and Gregory A Landrum. What’s what: The (nearly) definitive guide to reaction role assignment. *Journal of chemical information and modeling*, 56(12):2336–2346, 2016.
- [42] Srijan Shakya, Anamaria-Roberta Hartl, Sepp Hochreiter, and Korbinian Pöppel. Adaptive retrieval helps reasoning in llms – but mostly if it’s not used, 2026. URL <https://arxiv.org/abs/2602.07213>.
- [43] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>. GitHub repository.
- [44] Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, 2017.
- [45] Rich Sutton. The bitter lesson, 2019. URL <http://www.incompleteideas.net/IncIdeas/Bitter-Lesson.html>, 2019.

-
- [46] Sebastian Thrun and Lorien Pratt. Learning to learn: Introduction and overview. In *Learning to learn*, pp. 3–17. Springer, 1998.
- [47] Muxin Tian, Zhe Wang, Blair Yang, Zhenwei Tang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. Swe-bench mobile: Can large language model agents develop industry-level mobile applications? In *arXiv preprint*, 2026. URL <https://api.semanticscholar.org/CorpusID:285462974>.
- [48] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions, 2023. URL <https://arxiv.org/abs/2212.10509>.
- [49] Chenghao Xiao, G Thomas Hudson, and Noura Al Moubayed. Rar-b: Reasoning as retrieval benchmark, 2024. URL <https://arxiv.org/abs/2404.06347>.
- [50] Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs. In *OpenReview*, 2026. URL <https://api.semanticscholar.org/CorpusID:285454009>.
- [51] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [52] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.
- [53] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic “differentiation” via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- [54] Mert Yuksekgonul, Daniel Kocaja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to discover at test time, 2026. URL <https://arxiv.org/abs/2601.16175>.
- [55] Mert Yuksekgonul, Daniel Kocaja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.
- [56] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026. URL <https://arxiv.org/abs/2512.24601>.
- [57] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.
- [58] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation, 2025. URL <https://arxiv.org/abs/2410.10762>.
- [59] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, V. Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and K. Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *arXiv preprint arXiv:2510.04618*, 2025.
- [60] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification, 2016. URL <https://arxiv.org/abs/1509.01626>.

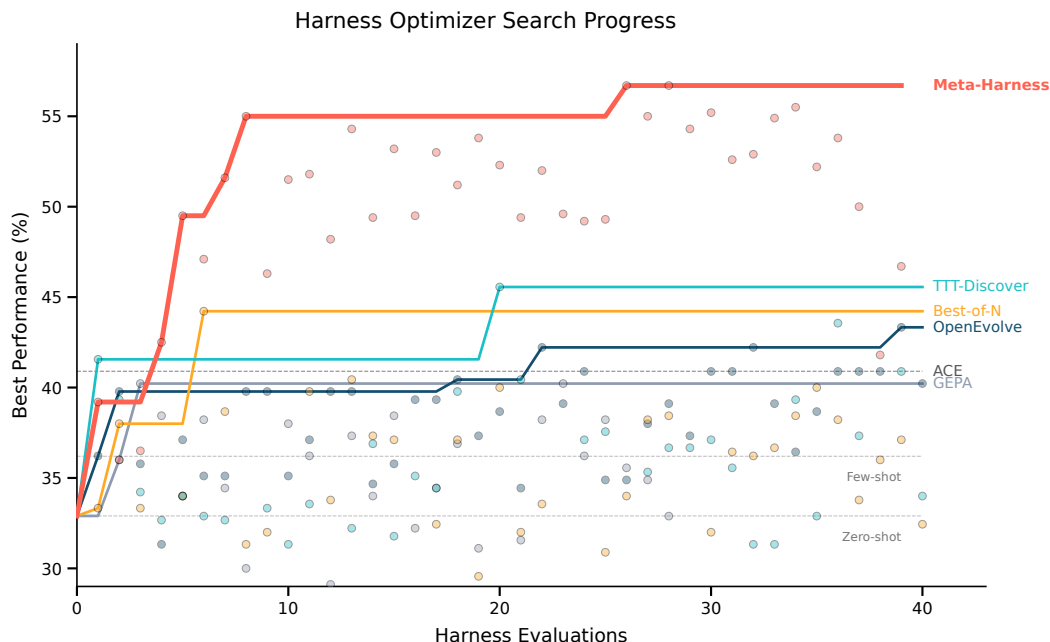


图 4：在线文本分类中所有对比文本优化器在评估中的搜索集准确率。每个点代表一个候选框架；线条跟踪迄今为止的最佳结果。每个数据集的曲线与聚合曲线一同展示。元框架在前 4 次评估内就达到了 OpenEvolve 和 TTT-Discover 的最终准确率，并持续改进，最终比所有基线高出 10 个百分点以上。

A 定性提议者行为

本节考察提议者在搜索过程中如何使用文件系统，基于 TerminalBench-2 运行（10 次迭代，Claude Opus 4.6）。

A.1 文件访问统计

为验证提议者实质性使用文件系统而非默认进行本地编辑，我们记录了每次迭代的所有文件读取。

表 8 总结了结果。提议者每次迭代读取文件的中位数为 82 个（范围 69–99），大致均匀地分布在先前测试框架源代码（41%）和执行轨迹（40%）之间，其余部分用于得分摘要（6%）和其他文件（13%）。这证实了提议者的访问模式是非马尔可夫的：它通常检查大部分可用历史，而不是仅以最近的父代为条件。

Statistic	Value
Files read per iteration (median)	82
Files read per iteration (range)	69–99
<i>File type breakdown</i>	
Harness source code	41%
Execution traces	40%
Score/summary files	6%
Other	13%

表 8：来自 TerminalBench-2 搜索运行（10 次迭代，Claude Opus 4.6）的提议者文件访问统计。提议者广泛读取文件系统，对先前源代码和执行轨迹的关注大致相等。

A.2 定性行为：对先前失败的因果推理

TerminalBench-2 搜索日志揭示了一个清晰的叙事弧线，其中提议者从自身的回归中学习。它不是在本地图编辑中随机游走，而是对早期候选者失败的原因形成明确的诊断，然后转向更安全的设计模式。下面日志框中的所有文本均逐字引自提议者在每次迭代中记录的推理（强调为我们所加）。

迭代 1 - 2：有前景的错误修复受到提示编辑的干扰。前两次迭代均将合理的结构性修复与提示模板修改捆绑在一起，且两者都从 64.4% 的终点站-基线急剧回退。迭代 1 针对泄露的终端标记导致的观测损坏，并添加了循环断路器：

假设：在长时间运行的任务中，命令结束标记片段会泄漏到大型语言模型的观测中，导致模型困惑并陷入无限的无工具调用循环。剥离这些标记并添加循环断路器将恢复被浪费的步骤。

该候选方案还引入了新的以清理为导向的提示模板和验证清单。迭代 2 提出了不同的状态机修复方案：

双重确认完成机制导致验证螺旋。在智能体提前解决任务但额外消耗 15 至 40 多个步骤进行重复验证的轨迹中观察到，因为每次验证命令都会重置待完成状态，需要另一个任务完成 → 清单 → 验证循环。

第二个候选方案完全移除了待完成机制，同时保留了标记剥离和新提示。它仍然回退，这给提出者带来了两个失败的候选方案，它们具有不同的结构变化但共享一个提示干预。

迭代 3：提出者识别出混杂因素。到迭代 3，提出者明确推断回退并非主要由于结构性错误修复本身：

Prior attempts: evo_marker_fix (58.9%, -5.6pp), evo_single_confirm (57.8%, -6.7pp)
--- both regressed. **Root cause of regressions: Prompt template changes (cleanup directives) caused the agent to delete necessary state before task completion.** The structural bugfixes were confounded with harmful prompt changes. evo_strip_only isolates the two proven structural fixes.

这是轨迹中的关键因果步骤。提出者注意到前两次失败的共同因素不是特定的错误修复，而是以清理为主的提示重写。因此，它恢复到原始提示，仅测试标记剥离和循环断路器。结果候选方案仍然略微表现不佳（63.3%，-1.1 个百分点），但损失远小于早期版本，这支持了混杂诊断。

迭代 4 至 6：对已诊断失效模式的直接修复仍导致性能回退。后续三次迭代继续探索设计空间的同一区域，但此时对完成逻辑脆弱的原因提出了更明确的理论。迭代 4 将失败归因于一个具体的状态机缺陷：验证命令会重置完成标志，使智能体陷入重复的检查清单循环：

Remove the two self.pending_completion = False lines that reset the completion flag when intermediate commands run. This fixes a state machine bug where: (1) Agent calls task_complete → sees QA checklist, pending_completion = True (2) Agent runs verification commands → pending_completion = False (bug!) (3) Agent calls task_complete again → sees checklist AGAIN → infinite loop.

提议者甚至引用了具体的轨迹证据，指出配置 Git 网页服务器（configure-git-webserver）在有效完成任务后，智能体仍陷入 30 至 60 步的验证循环，导致基线失败。迭代 5 试图弱化清理措辞，同时保留

确认机制，但仍修改了提示词并严重回退。迭代 6 回归到更安全的仅剥离基线的演进方式，并提出系统级优化：

空命令轮次在终端输出未变化时浪费了完整的大语言模型往返。智能等待（在下次大语言模型调用前轮询窗格最多 3×5 秒）可在长时间运行任务中节省 5 至 15 个轮次。

该改动同样导致回退。至此，提议者已获得一条具体的经验教训：即使局部假设听起来合理，对提示词和完成流程的修改仍属高风险操作。

迭代 7：胜出候选方案。在连续六次回退后，提议者将策略从修改控制循环转向在循环开始前增加信息：

此前全部 6 次迭代均从 64.4% 的基线回退，原因是它们修改了补全流程、提示模板或观测处理。evo 环境引导则采取了一种不同的方法——**纯增量式**。它在首次大语言模型调用前通过单条 shell 命令收集环境快照，并将其附加到初始提示中。其他方法均未改动。这应能在依赖繁重的任务上消除 3 至 5 次浪费的探索轮次，同时避免在已通过任务上产生回退风险。

该候选方案是迄今最佳结果。重点不仅在于第 7 次迭代胜出，更在于提出者阐明了其为何更安全：它避免触及此前脆弱的补全机制，而是添加了主要对困难任务有用的信息。

第 8 次迭代：组合。在找到一项增量式改进后，提出者接下来尝试将其与此前的一项结构性修复相结合：

```
Combining two orthogonal fixes --- env snapshot (saves early exploration turns) +
marker stripping with no-tool-call loop breaker --- will yield +1--3pp because they
address independent failure modes without touching prompts or confirmation flows
(which caused regressions in 5 of 7 prior iterations).
```

第 10 次迭代：跨运行迁移。提出者引用了来自另一独立早期搜索运行的结果：

演化历史显示，“不清理服务工件”这一做法带来了 +18 个百分点的提升。第 9 次迭代（无清理指令的演化）针对同一思路，但在评估前崩溃。

Summary

搜索轨迹表明，提议者所做的不仅仅是随机变异。在前七次迭代中，它识别出一个混杂因素，直接测试了隔离该混杂因素的假设，观察到控制流和提示编辑仍然脆弱，然后有意转向一种纯增量式修改，该修改成为本次运行中的最佳候选。随后，它尝试将该获胜思路与早期修复相结合，甚至在不同运行之间迁移经验。这种基于先前失败的因果推理，正是完整历史文件系统访问所支持的，也是压缩反馈优化器无法实现的。

B 发现的测试框架

元测试框架（Meta-Harness）能够发现针对当前问题设置的可执行推理时程序。这些测试框架是结构化的、领域特定的策略，通常具有非平凡的控制流，如路由、过滤和条件上下文构建，其选择完全取决于它们是否提升搜索集性能。本节以紧凑的、方法风格的抽象形式呈现代表性测试框架，概括驱动推理时行为的主要行为和控制流决策。作为参考，每个已发现测试框架的完整实现规模约为 100 – 1000 行代码。

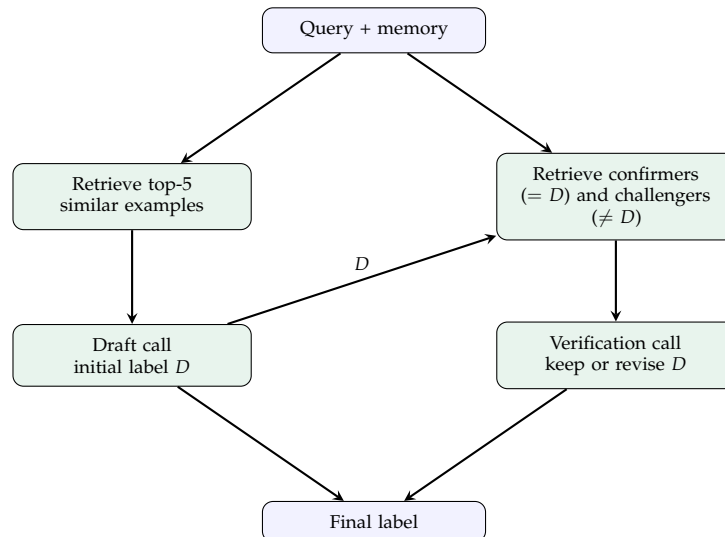


图 5：草稿-验证分类测试框架。第一次调用从简短的检索上下文中生成草稿标签。第二次调用检索支持和反对该草稿的证据，并返回最终预测。

B.1 文本分类测试框架

在在线文本分类中，元测试框架发现的是一族基于记忆的测试框架，而非单一规范策略。表 9 报告了主搜索中非支配变体的帕累托前沿，所有变体均仅根据搜索集性能进行选择。我们在此强调两个代表性端点：元测试框架（草稿验证），即最低上下文的前沿点，以及元测试框架（标签引导查询），即正文中使用的最高准确率前沿点。

概述。两种框架均持续维护已标注样本的记忆库，并在推理时基于该记忆构建提示。区别在于查询记忆的控制流不同。元框架（草稿验证）采用两次短调用，并显式地将模型的首次猜测与检索到的反例进行比对；而元框架（标签引导查询）则将更大的单次调用预算用于显式呈现标签空间与局部决策边界。图 5 和图 6 总结了这两种程序。

元框架（草稿验证）。对应的发现文件为 `draft verification.py`。该轻量级变体将预测转化为两阶段调用过程。首先检索 5 个最相似的已标注样本并生成草稿预测。随后基于该草稿标签重新查询同一记忆库，检索 5 个同标签的确认样本和 5 个不同标签的挑战样本，并询问模型是维持还是修改初始答案。发现的关键行为是第二次检索同时依赖于查询和草稿预测，因此该框架能够针对模型当前猜测呈现有针对性的反例，而非仅提供通用的近邻样本。若已积累的标注样本过少，程序则回退至标准的单次调用少样本提示。

- **阶段 1：草稿。**检索 5 个最近邻已标注样本并请求初始预测。
- **阶段 2：验证。**基于草稿标签进行条件检索，然后同时展示支持性和挑战性样本，再做出最终预测。
- **冷启动。**如果可用的标注样本少于 5 个，则跳过两阶段流程，改用标准的单次调用少样本提示。
- **为何成本低廉。**两次调用均使用较短的检索上下文，因此即使进行两次模型调用，整体上下文成本仍保持在效率前沿的低端附近。

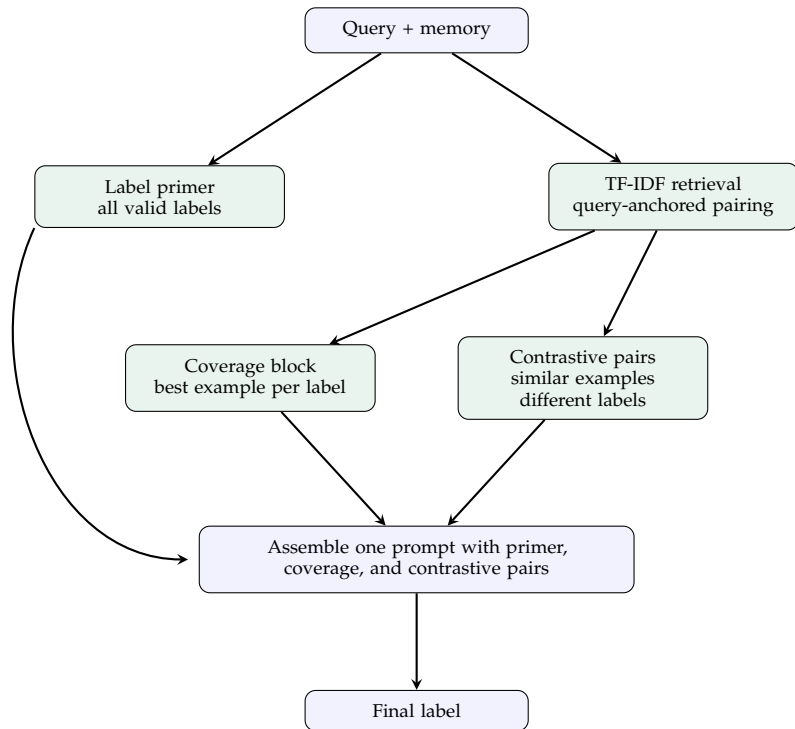


图 6：标签引导的查询锚定分类框架。该程序构建一个单一提示以展示标签空间，然后填充与查询相关的覆盖示例和局部对比样本对。

元框架（标签引导查询）。对应的发现文件为标签引导查询锚定.py。这一最强变体使用由三部分组成的单次较大调用。首先以标签引导语列出有效输出标签，然后构建覆盖部分，为每个标签提供一个与查询相关的示例，最后添加查询锚定的对比样本对，将标签不同但高度相似的示例并排呈现。覆盖块展示完整标签空间，而对比块则锐化当前查询周围的局部决策边界。在代码中，该框架通过对历史标注示例进行 TF-IDF 检索来实现，并采用查询锚定的配对规则，从同一局部邻域中选择形成对比的示例。

- **标签引导语。**在展示任何示例之前先列出有效输出标签，使模型预先看到完整的答案空间。
- **覆盖块。**对于每个已知标签，检索与查询最相关的已标注示例，并为每个类别包含一个代表性示例。
- **对比块。**构建标签不同但高度相似的示例对，使提示暴露当前查询周围的局部决策边界。
- **检索规则。**使用词频-逆文档频率（TF-IDF）相似度和以查询为锚点的配对选择，而非标签无关的最近邻。

B.2 数学检索测试框架

本小节描述元测试框架（Meta-Harness）为数学推理发现的检索测试框架（第 4.2 节）。最终框架是一个紧凑的四路 BM25 程序，其结构是通过搜索涌现的，而非事后人工指定。以下所有设计选择——路由谓词、重排序项、去重阈值以及每路示例数量——均由外层循环在 40 次进化迭代中选定。

Variant	Datasets			Avg metrics	
	USPTO $\uparrow$	Symptom $\uparrow$	LawBench $\uparrow$	Avg $\uparrow$	Ctx $\downarrow$
Meta-Harness (Draft Verification)	18.0	85.4	17.0	40.1	5.4
Meta-Harness (Error-Annotated)	9.0	87.7	24.0	40.2	22.3
Meta-Harness (CoT Replay)	13.0	88.2	25.0	42.1	23.3
Meta-Harness (Cluster Coverage)	12.0	86.8	33.0	43.9	31.2
Meta-Harness (Cascade Retrieval)	12.0	86.8	36.0	44.9	39.2
Meta-Harness (RRF + Contrastive)	18.0	89.6	35.0	47.5	41.4
Meta-Harness (Relevance + Contrastive)	18.0	90.6	36.0	48.2	43.9
Meta-Harness (Label-Primed Query)	14.0	86.8	45.0	48.6	45.5

表 9：主文本分类搜索中发现的帕累托最优变体，在平均准确率与上下文成本之间进行权衡。正文中选定的系统为元框架（标签引导查询）。上下文成本表示输入上下文中平均额外字符数（千）。

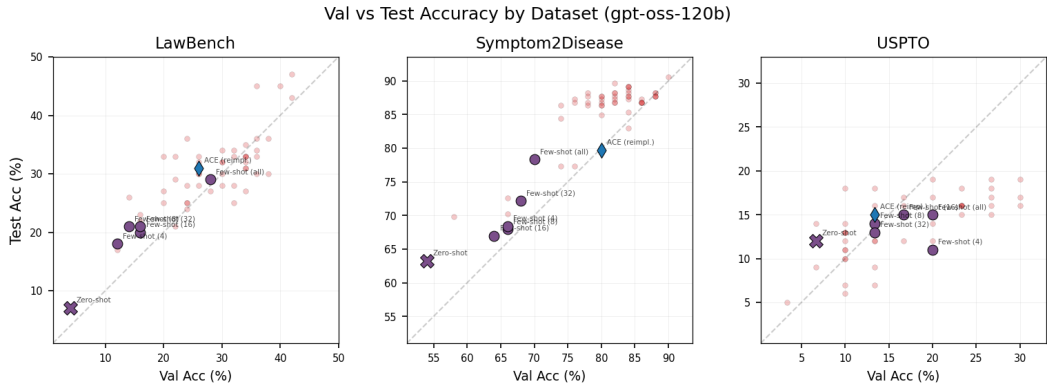


图 7：发现的文本分类策略在各数据集上的搜索集与测试准确率对比。每个粉色圆点代表一种发现的策略；基线已标注。虚线对角线为 $y=x$ 。

概述。在推理阶段，该框架将每个问题精确分配给四条路径之一：组合数学、几何、数论，或针对代数及其他问题的默认路径。门控机制实现对问题陈述的轻量级词汇谓词，包括关键词集合以及少量针对几何符号的正则表达式特征。该框架不会跨路径聚合输出：一旦选定路径，仅由该路径检索示例用于最终提示。所有路径均使用 BM25 作为上述过滤语料库上的底层检索机制。BM25 索引采用数学感知的分词器，将 LaTeX 标记（例如 $\frac{1}{2}$ ）保留为原子单元。选定的框架是两条成功搜索谱系的合并，由提议者在搜索过程中自主组合：一条贡献了基于原始 BM25 的更强几何路径，另一条贡献了基于去重和难度重排的更强组合数学路径。图 8 给出了最终程序的紧凑流程图视图。

- **组合数学：**获取 20 个 BM25 候选，去重至 8 个，按词汇得分和难度重排，然后返回前 3 个。这是该框架明确在多样性与难题匹配之间进行权衡的主要路径。

- **几何：**返回 1 个困难的 NuminaMath 参考以及 2 个原始 BM25 邻居。搜索在此处始终偏好原始结构匹配而非难度重排。

- **数论：**获取 12 个 BM25 候选结果，并根据词汇得分、难度以及一个对在早期陈述技巧的解法给予的小额奖励进行重排序。这有利于那些证明策略明确的示例。

- **默认：**获取 10 个 BM25 候选结果，根据词汇得分和难度进行重排序，并根据最高检索得分的集中程度自适应地选择示例数量。

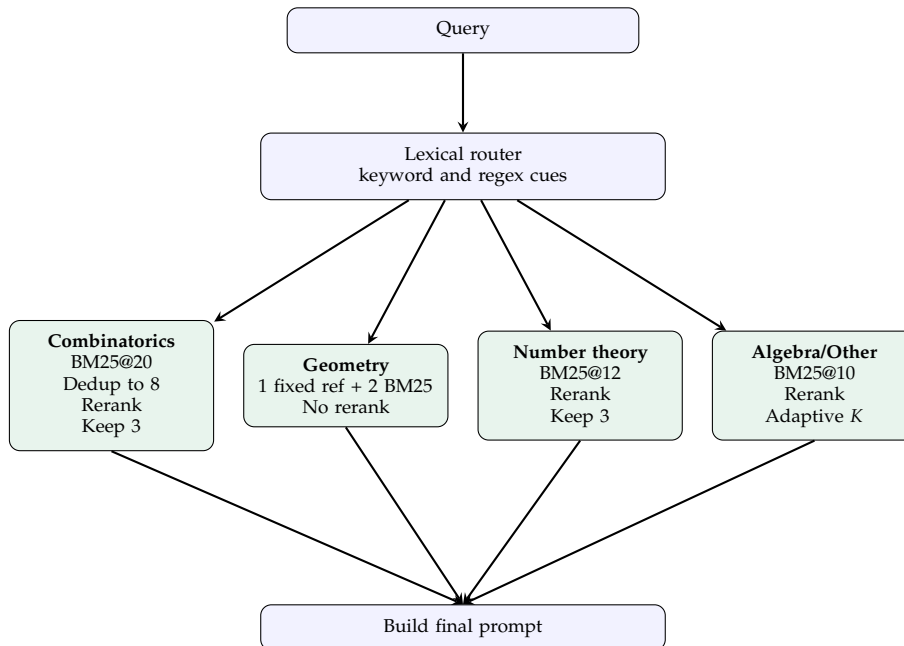


图 8：发现的数学检索框架。一个词汇路由器将每个查询分配给四个学科特定的检索策略之一。所选策略检索示例，并将其插入最终提示中。

B.3 TerminalBench-2 框架

发现的 TerminalBench-2 框架建立在 Terminus-KIRA [25] 之上，继承了其原生工具调用（取代 Terminus 2 基于 ICL 的 JSON 解析）、30KB 输出上限以及多视角完成清单。Meta-Harness 发现的主要修改是

环境引导：在智能体循环开始之前，该框架运行一个复合 shell 命令来收集沙箱环境的快照，并将其注入初始提示中。提议者的假设，逐字记录自搜索日志，为：

Hypothesis: ‘‘Injecting an environment snapshot (OS, installed languages, package managers, /app contents) before the first LLM turn will reduce wasted exploration episodes by 3--5 turns on dependency-heavy tasks’’

Changes: ‘‘Added `_gather_env_snapshot()` that runs a single compound shell command to collect working directory, /app listing, available languages (python, gcc, node, java, rustc, go), package managers (pip, apt) [...] and injects as [Environment Snapshot] block’’

快照包括：工作目录、/app 的目录列表（大目录截断为 20 个条目）、可用的编程语言及其版本（Python、GCC、G++、Node、Java、Rust、Go）、已安装的包管理器（pip、apt-get）以及可用内存。这消除了智能体通常花费 2–4 个探索轮次来发现可用工具和文件的过程，使模型能够立即开始高效工作。引导命令受 15 秒超时保护且静默失败，因此在异常环境中不会中断智能体。完整实现在 Terminus-KIRA 之上增加了约 80 行代码。图 9 总结了测试框架结构。

逐任务分析。与 Terminus-KIRA 相比，发现的测试框架在 89 个任务中的 7 个上有所提升，其中蛋白质组装和路径追踪任务提升最大。这些提升的任务具有共同特性：它们需要领域专用工具，而这些工具的可用性无法预先假设（生物信息学库、渲染管线、国际象棋引擎、密码学工具、CoreWars 模拟器）。没有引导时，智能体花费其

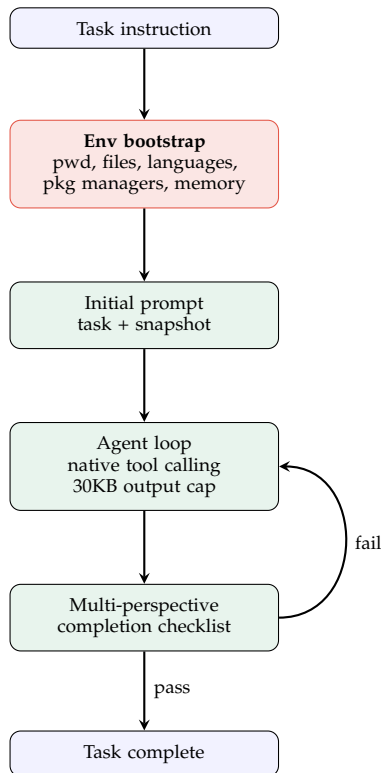


图 9：发现的 TerminalBench-2 测试框架。该框架继承了 Terminus-KIRA 的原生工具调用、输出上限和完成清单（绿色）。环境引导（红色）是 Meta-Harness 发现的组件：它在智能体循环开始前收集沙箱快照，消除了早期探索轮次。

前 2-4 个轮次探测环境：在轮次预算紧张或早期错误假设级联的任务中，这些浪费的轮次可能决定通过与否。这表明当环境不明显且任务要求智能体根据实际安装内容调整策略时，引导的价值最大。

C 数据集详情

C.1 分布外文本分类数据集

- **SciCite** 是由 Cohan 等人提出的三分类引用意图分类基准 [14]。每个样本包含来自科学论文的引用上下文，并标注引用的修辞作用，如背景、方法或结果。该任务测试模型能否从局部科学上下文推断一篇论文引用另一篇论文的原因。

- **FiNER-139** 是由 Loukas 等人提出的金融数字实体识别基准 [29]。它由来自金融文件的词级标注组成，包含 139 种细粒度 XBRL 实体类型，使其比标准的句子级分类任务更加细粒度。该基准测试模型能否从上下文中识别和分类数字金融实体。

- **亚马逊评论** 是 Keung 等人提出的多语言亚马逊评论语料库的英文部分 [22]。在我们的设置中，它被用作一个五分类的评论评分预测任务，其中标签对应于评论的星级评分。该基准评估了从产品评论文本中进行通用领域情感和评分预测的能力。

- **金融短语库** 是由马洛等人提出的三分类金融情感基准。[32] 它由金融新闻及相关经济文本中的句子组成，并带有标注。

分为积极、中性或消极，以反映市场情绪。该任务评估金融领域特定情感分类。

- **情感粒度基准 (GoEmotions)** 是由德姆斯基等人提出的细粒度情感分类基准 [15]。该基准包含标注了 27 种情感类别及一个中性类别的英文红迪网评论，通常被视为 28 类分类任务。该基准测试的是超越粗粒度正负情感判断的细腻情感识别能力。
- **银行 77 (Banking77)** 是卡萨努埃瓦等人提出的细粒度意图分类基准 [11]。它包含标注了 77 种意图的在线银行用户话语，覆盖广泛的客户服务请求。该任务评估具有大标签空间的单领域意图检测。
- **AG 新闻 (AG News)** 是一个四类新闻主题分类基准，通常与 Zhang 等人的文本分类设置相关联 [60]。样本被标注为广泛的新闻类别，如世界、体育、商业和科学/技术。它是主题分类的标准通用领域基准。
- **SciTail** 是一个科学领域的文本蕴含基准，其任务是在以科学为重点的推理设置中，预测一个假设是否被一个前提句所蕴含 [24]。

• **TweetEval (仇恨)** 是 Barbieri 等人提出的 TweetEval 基准中的仇恨言论子集 [7]。它是一个二分类推文分类任务，用于在统一的社交媒体评估套件中检测仇恨与非仇恨内容。该基准测试了在嘈杂、短形式的社交媒体文本中的鲁棒分类能力。

C.2 数学检索语料库

表 10 列出了第 4.2 节中使用的检索语料库所包含的数据集。原始来源包含的问题比最终语料库更多；在合并之前应用了多个过滤步骤。NuminaMath-1.5 被过滤为竞赛数学子集 (AMC/AIME、奥林匹克参考、数论、不等式及相关来源)，丢弃了低质量的网络抓取条目。OpenMathReasoning 被去重为每个问题一个解答（保留在独立验证器上通过率最高的解答），并且在去重之前，移除了来源与任何评估基准家族 (IMO、AIME、HMMT、SMT、USAMO、Putnam) 匹配的问题。随后，整个语料库针对所有评估基准和工具搜索期间使用的搜索集进行了去污染处理，使用精确前缀匹配后接模糊 Jaccard 相似度（阈值 0.8）；在任一标准下与评估问题匹配的语料库问题均被丢弃。来自 OpenMathReasoning 和 DeepMath 的解答被截断为 5000 个字符，以限制检索上下文长度。在运行时，选定的工具进一步将检索限制为解答非空且长度小于 4000 字符的条目。检索到的解答在插入提示时再次被截断为 3000 个字符。对于几何路线，该工具还从难度大于 6 的 NuminaMath 问题中构建了一个单独的硬参考索引。

C.3 数学 IMO 级别测试集

正文汇总了来自国际数学奥林匹克竞赛答案基准 (IMO-AnswerBench)、国际数学奥林匹克竞赛证明基准 (IMO-ProofBench)、2025 年 12 月 ArXivMath 和 2026 年 1 月 ArXivMath 的 200 道国际数学奥林匹克竞赛级别问题的结果。200 道问题的评估集由国际数学奥林匹克竞赛答案基准中分层抽取的 100 道问题子集，以及其余三个基准的全部问题组成。这种按基准划分的细目很有用，因为四个数据集混合了答案型、证明型和研究型问题，在正文中为简洁起见将其合并。若包含本节的表格，应分别报告五个留出模型在基础 (Base) 和元测试框架 (Meta-Harness) 下每个基准的结果。

Dataset	Problems	Sol. Len	Proof
OpenMathReasoning	281,743	5,000 [†]	34%
DeepMath-103K	103,021	5,000 [†]	0%
NuminaMath-1.5	129,520	1,376	13%
PolyMath	11,083	363	0%
Omni-MATH	4,289	829	0%
FineProofs-SFT	4,275	3,977	100%
AIME 1983–2024	933	—	0%
Putnam-AXIOM	492	888	100%
Total	535,356	5,000[†]	22%

[†] 截断至 5,000 字符；实际解答更长。

表 10: 数学检索语料库中的数据集中数据集（共 535K 道问题）。解答长度（Sol. Len）为解答长度的中位数（以字符计）。证明（Proof）表示数据集是否包含证明型问题（根据答案或问题类型字段）。

Dataset	Problems
IMO-AnswerBench	100
IMO-ProofBench	60
ArXivMath Dec. 2025	17
ArXivMath Jan. 2026	23
Total	200

表 11: 200 道国际数学奥林匹克竞赛级别评估集的细目。

D 实用实现技巧

元测试框架（Meta-Harness）在很大程度上与领域无关：我们预期它适用于任何由任务特定测试框架包装语言模型的环境。然而，将其应用于新领域需要在相对新颖的大语言模型辅助编程模式下运行，在该模式下，提议者以先前运行的长期历史为条件，编写其效果可能在许多步骤之后才显现的程序。为使该工作流程可靠运行，我们发现了一小组在本文研究的三个领域中始终重要的实用选择。以下指南本身并非关于该方法的科学论断；它们是构建和运行系统的工程经验，我们希望这有助于未来工作在其他领域应用元测试框架。

- **编写良好的技能。**技能文本是引导搜索的主要接口，其质量是决定循环能否有效运作的最强杠杆。提议者接收一段自然语言技能 [5]，该技能定义了其角色、目录布局、命令行命令及输出格式。实践中，技能应约束输出及安全相关行为，而非提议者的诊断流程：它应明确禁止事项、需产出的工件以及需优化的目标，同时允许模型根据需要自由检查分数、迹及先前代码。根据我们对元基准运行日志的检查，直觉是经过足够多的迭代后，累积的迹对提议者行为的影响往往超过技能本身。根据我们的经验，迭代技能文本对搜索质量的影响大于改变迭代次数或种群规模。建议先进行几次简短的演化运行（每次 3–5 次迭代），专门用于调试和完善技能，然后再投入完整运行。

- **从基线框架和一个对其而言困难的搜索集开始。**编写一个简单的基线（例如少样本提示），然后通过筛选基线出错的示例或选择困难实例的多样化子集来构建搜索集。如果基线已经使评估饱和，则搜索几乎没有优化空间。保持搜索集足够小，以便每次运行大约进行 50 次完整评估（在我们的实验中为 50–100 个示例）

分类实验，88 个数学检索问题）；快速且具有区分度的评估比大规模评估更有价值。

- **以易于浏览的格式记录所有内容。**评估代码应以提议者可可靠查询的形式编写代码、分数和执行迹。在实践中，这意味着使用机器可读格式（如 JSON），分层组织工件，选择合理且一致的文件名，并采用使简单工具（如正则表达式搜索）能够良好工作的命名方案。

- **通过一个小型命令行界面（CLI）使日志可查询（可选，但有用）。**每个测试框架（harness）都有一个目录，包含源代码、评分和执行轨迹，但随着历史记录的增长，仅靠原始文件系统访问变得繁琐。一个简短的命令行界面（CLI）可以列出帕累托前沿、显示前 k 个测试框架，并对比成对运行之间的代码和结果差异，这能让经验存储更易于使用，而查询此类 CLI 与编码代理所训练的工作流程高度一致。如果存在相关的离线经验（来自其他模型的展开、已解决问题的语料库、相关论文），将其转换为相同的目录结构也有助于热启动探索并奠定新思路的基础。这一层帮助提议者节省可能浪费在导航上的令牌。

- **在昂贵的基准测试之前进行轻量级验证。**编写一个小型验证测试，导入模块、实例化类，并在一小组示例上调用两个方法。搜索过程中提出的测试框架应在完全评估之前通过此测试。一个简单的测试脚本可以在几秒钟内捕获大多数格式错误或不可用的候选方案，并将失败成本保持在接近零的水平。

- **在提议者之外自动化评估。**运行评估足够简单，不值得让提议者来做。一个独立的测试框架应对候选方案进行评分并将结果写入文件系统。

E 扩展相关工作

本附录扩展了第 2 节中的简要讨论，并将元测试框架（Meta-Harness）相对于几条相邻工作线进行定位，这些工作线在主文中无法详细覆盖。一个反复出现的区别是，元测试框架优化可执行的测试框架实现，并通过文件系统为提议者提供对先前代码、评分和执行轨迹的选择性访问。

AlphaEvolve / OpenEvolve。AlphaEvolve [35] 与 OpenEvolve [43] 通过大语言模型引导的变异与结构化反馈来演化代码：提议者接收一个带有标量评分的程序数据库（每步 4 – 22K 词元；表 1），并对锦标赛选择的父代应用固定的变异策略。这些方法专为算法发现与优化（数学猜想、调度启发式、硬件核函数）而设计，其中搜索目标是一个单一的无状态函数，具有清晰的标量目标，且变异是局部的。测试框架工程则属于另一种范畴：测试框架是有状态程序，会在众多示例中积累经验，而单一设计选择（例如在内存中存储什么）可能贯穿整个评估序列。元测试框架通过赋予非结构化编码智能体完整的文件系统访问权限来解决这一问题，使其能够有选择地读取任何先前候选者的源代码、执行轨迹和评分。

GEPA。GEPA [1] 在反馈丰富度方面是最接近的文本优化器，为每个候选者提供执行轨迹。它专为具有短反馈循环的任务（数学问题、指令遵循、代码优化）中的提示优化而设计，其中每次执行是单次大语言模型调用或一个短流水线。在这种模式下，逐候选者的反思效果良好：一个提示、一个答案、一个评分。测试框架工程需要同时跨多个示例和多个候选者进行推理：理解为何某个检索策略对一类问题有效而对另一类问题退化，需要比较整个种群中的执行轨迹。GEPA 一次仅处理一个候选者（每步 2 – 8K 词元；表 1），其固定的批评格式必须预先判断哪些信息是相关的。元测试框架让提议者同时访问所有先前候选者，并让智能体自行决定要检查什么。

提示编排框架。多个系统为组合多阶段大语言模型程序提供了结构化抽象。LMQL [8]、LangChain [13] 和 DSPy [23] 通过为提示模板、控制流和模块化大语言模型流水线提供更高层次的接口，使提示工程更加系统化。这些框架帮助开发者指定和组织大语言模型程序，但它们通常仍需要手动设计检索策略、记忆更新和编排逻辑。元框架（Meta-Harness）在不同的层面上运作：它在可执行代码中搜索这些策略的实现，将框架本身视为优化目标。